

# Sessions verstehen und anwenden

10 kleine Aufgaben · Startcode mit TODOs · Lösungen getrennt

## Das Ziel

Den bekannten Formularablauf um einen serverseitigen Zustand erweitern: Eingaben prüfen, in der Session merken und auf weiteren Seiten verwenden. Danach Meldungen einmalig anzeigen und die Session bewusst beenden. Keine Datenbank und kein echter Login.

**Browser und Server:** Das Cookie enthält hier die Session-ID. Die fachlichen Werte liegen auf dem Server und werden durch `session_start()` in `$_SESSION` geladen. [1, 2]

## Start in XAMPP

Lege das gesamte Repository `PHP_AI_DEMO` unter `C:\xampp\htdocs` ab und starte Apache. Öffne die Kursübersicht unter `http://localhost/PHP_AI_DEMO/` und wähle den passenden Arbeitsbereich.

Einstieg im Repository: `PHP_Arbeitsbereich/session/index.html`

## Die bekannte Dateiaufteilung

**index.php:** Session starten, Formular und Zustand anzeigen.

**auswertung.php:** Session starten, POST prüfen, speichern und zurückleiten.

**function.php:** vorgegebene Konfiguration und HTML-Ausgabe; startet noch keine Session. Einzelne Aufgaben ergänzen eine zweite Seite.

## Für jede Aufgabe verbindlich

1. Vor Session-Zugriff und vor jeder Ausgabe die Session starten. Name und Cookie-Pfad sind pro Aufgabe vorgegeben. [2, 6]
2. Jeden POST-Eintrag mit `isset()` und `is_string()` prüfen; danach `trim()` und die Inhaltsprüfung. Bei Session-Werten den passenden Typ prüfen: Text, int oder Array.
3. Bei mehreren Feldern erst alle prüfen, dann speichern. Ein Fehler verändert keine gespeicherten Nutzdaten. Gültige neue Formwerte erhalten, ungültige leeren.
4. Nach einer gültigen POST-Aktion mit `header()` und HTTP 303 weiterleiten; danach `exit`. F5 auf der Zielseite darf die Aktion nicht erneut ausführen. [8, 9]

## Der Rückweg wächst mit

Aufgaben 1–7 nutzen für Formfehler noch den bekannten GET-Rückweg. Aufgabe 7 ergänzt eine einmalige Session-Meldung. Ab Aufgabe 8 kommen auch Fehler und gültige Eingaben über die Session zurück. Verwende ausschließlich kurze, erfundene Daten.

# 01 Einen Alias merken

## POST · alias (Text)

Session starten, speichern und auf einer zweiten Seite lesen

1. Ergänze `session_start()` vor jeder Ausgabe in `index.php`, `auswertung.php` und `anzeige.php`. Die Konfiguration in `function.php` ist vorgegeben.
2. Empfange `alias` mit POST. Prüfe `isset()`, Texttyp und den nach `trim()` verbleibenden Inhalt. Leer ist ungültig; ein Fehler verändert keinen gespeicherten Alias.
3. Speichere einen gültigen Alias in `$_SESSION` und leite mit HTTP 303 zu `index.php` zurück. Dort wird er geprüft gelesen und angezeigt.
4. Ergänze `anzeige.php`: Alias ausschließlich aus der Session lesen. Ohne Eintrag „Noch kein Alias“ anzeigen. Kein Alias in URL oder verstecktem Feld.

## Teste selbst

Pixel speichern → beide Seiten zeigen Pixel; F5 erhält den Wert.

Leer oder `alias[]=Pixel` → Fehler; bisheriger Alias bleibt gespeichert.

Zweites Browserprofil → eigener leerer Stand; weiterer Tab desselben Profils teilt den Stand.

# 02 Eine Einstellung verändern

## POST · modus (lesen / ueben)

Einen vorhandenen Session-Wert überschreiben

1. Starte die Session auf beiden PHP-Seiten. Lies einen vorhandenen Lernmodus geprüft ein; ohne gültigen Eintrag gilt lesen.
2. Prüfe den POST-Eintrag `modus` ausdrücklich auf Vorhandensein, Texttyp und die erlaubten Werte lesen oder ueben.
3. Speichere nur einen gültigen Modus unter demselben Session-Schlüssel. Leite danach zurück; Anzeige und Vorauswahl müssen den neuen Wert zeigen.
4. Ein fehlender, leerer oder fremder Wert erzeugt eine Fehlermeldung. Das Fehlerfeld bleibt leer; ein älterer gespeicherter Modus bleibt unverändert.

## Teste selbst

Lesen speichern, dann Üben → Üben ersetzt Lesen.

`modus=spielen` → Fehler und leere Auswahl; alter Modus bleibt gespeichert.

F5 nach dem Speichern → derselbe Modus.

## 03 Nur einen Eintrag entfernen

POST · notiz (Text) · Aktionen speichern / löschen

`unset()` löscht gezielt einen Schlüssel

1. Starte die Session. Initialisiere einen fehlenden Bereich einmalig mit `werkraum` und zeige Bereich sowie Notiz getrennt an.
2. Prüfe beim Speichern die Notiz mit `isset()`, `is_string()` und `trim()`. Ein leerer Text darf einen alten Eintrag nicht überschreiben.
3. Beim POST-Auftrag löschen entfernst du ausschließlich die Notiz. Bereich und Session müssen erhalten bleiben. Löschen verlangt keine neue Texteingabe.
4. Leite nach jeder gültigen Aktion zurück. Ohne Notiz erscheint „Keine Notiz“; erneutes Löschen darf keine PHP-Warnung auslösen.

### Teste selbst

„Schlüssel abholen“ speichern und löschen → nur die Notiz verschwindet.

Der Bereich `werkraum` bleibt nach dem Löschen sichtbar.

Löschen bei leerem Feld → erlaubt; Speichern bei leerem Feld → Fehler.

## 04 Punkte sammeln – auch null

POST · punkte (ganze Zahl 0 bis 5)

Session-Zahl verändern und erneute POST-Ausführung vermeiden

1. Starte die Session. Lies den bisherigen Punktestand als ganze Zahl; ohne gültigen Stand gilt 0.
2. Prüfe `punkte` aus POST: vorhanden, Texttyp, nach `trim()` nur Ziffern und als Zahl von 0 bis 5. Leer ist ungültig; 0 ist ausdrücklich erlaubt.
3. Addiere nur eine gültige Eingabe zum bisherigen Stand. Speichere das Ergebnis als `int` in der Session und leite anschließend per 303 zurück.
4. Ein fehlerhafter Aufruf verändert den Punktestand nicht. Das erneute Laden der zurückgeleiteten GET-Seite darf keine Punkte erneut addieren.

### Teste selbst

2 und danach 3 senden → Punktestand 5.

0 senden → gültig, Punktestand unverändert.

F5 → kein erneutes Addieren; leer, -1, 6 und 2.5 → Fehler.

## 05 Zwei Einstellungen gemeinsam merken

POST · sprache (de / en) · schrift (normal / gross)

Ein assoziatives Array in der Session speichern

1. Prüfe beide POST-Einträge vollständig: vorhanden, Texttyp und erlaubter Inhalt. Sammle Fehler und gültige neue Eingaben getrennt.
2. Bei Fehlern bleibt die gespeicherte Einstellung unverändert. Nur gültige neue Formwerte werden wieder eingesetzt; ungültige Felder bleiben leer.
3. Erst bei zwei gültigen Werten speicherst du beide in einem assoziativen Array unter `einstellungen` in `$_SESSION`. Leite danach zurück.
4. Prüfe beim Lesen zunächst das Array und dann seine Einträge. Ohne gültige Einstellungen gelten `de` und `normal`. Zeige gespeicherten Stand und Formular getrennt.

### Teste selbst

en / gross → beide Werte bleiben nach erneutem Aufruf erhalten.

en / leer → Englisch im Formular, Schrift leer; bisheriger Speicherstand unverändert.

Beide leer → Meldungen zu beiden Feldern.

## 06 Eine kleine Merkliste führen

POST · begriff (Text) · Aktionen hinzufuegen / leeren

Ein Session-Array erweitern und durchlaufen

1. Prüfe den POST-Begriff mit `isset()`, `is_string()` und `trim()`. Ein leerer Begriff ist ungültig. Lies die vorhandene Merkliste als Array.
2. Erlaube höchstens fünf Einträge und keine identischen Doppelungen. Groß- und Kleinschreibung werden dabei unterschieden. Bei Fehlern bleibt die Liste unverändert.
3. Hänge nur einen gültigen neuen Begriff an. Speichere die Liste in der Session, leite zurück und gib ihre Einträge mit `foreach` sicher in HTML aus.
4. Die POST-Aktion `leeren` entfernt nur die Merkliste und benötigt keine neue Texteingabe. Nach der Rückleitung erscheint „Noch keine Begriffe“.

### Teste selbst

PHP, dann HTML → beide Begriffe in der Liste.

PHP erneut → Fehlermeldung, keine Doppelung; sechster Begriff → abgelehnt.

F5 dupliziert nichts; Leeren entfernt die Liste.

## 07 Eine Meldung nur einmal anzeigen

POST · notiz (Text)

Flash-Meldung: lesen, merken, aus der Session entfernen

1. Prüfe eine neue Notiz wie bisher. Bei gültigem Text speicherst du die Notiz in der Session; bei Fehlern bleibt der alte Wert erhalten.
2. Lege bei Erfolg zusätzlich die Meldung „Notiz gespeichert.“ in einem eigenen Session-Eintrag ab. Leite mit HTTP 303 zum Formular zurück.
3. Lies die Meldung in index.php in eine lokale Variable und entferne anschließend nur ihren Session-Eintrag. Gib die lokale Meldung unter dem Formular-Titel aus.
4. Lade die Seite erneut: Die Meldung ist weg, die Notiz bleibt. Eine weitere erfolgreiche Speicherung erzeugt wieder genau eine neue Meldung.

### Teste selbst

„Buch zurückgeben“ speichern → Notiz und Meldung sichtbar.

F5 → Meldung verschwunden; Notiz weiterhin gespeichert.

Leere Eingabe → Fehler, keine neue Erfolgsmeldung.

## 08 Fehler und gültige Eingaben zurückgeben

POST · artikel (Text) · anzahl (ganze Zahl 0 bis 9)

Den bisherigen GET-Rückweg durch Session-Daten ersetzen

1. Prüfe beide POST-Felder unabhängig voneinander. Artikel ist Pflicht; Anzahl muss ganz und von 0 bis 9 sein. Sammle alle Fehler und nur gültige neue Werte.
2. Bei Fehlern: Meldungen und gültige Formwerte getrennt in der Session ablegen. Mit HTTP 303 zu index.php zurückleiten – ohne Formwerte oder Fehler in der URL.
3. Lies diese Rückgabedaten einmalig aus und entferne sie danach aus der Session. Gültige neue Felder bleiben gefüllt, ungültige leer. Alte gespeicherte Werte nicht hineinmischen.
4. Erst bei zwei gültigen Eingaben die Vormerkung speichern. Bei fehlerhaften Versuchen bleibt eine ältere Vormerkung unverändert. Es wird nichts bestellt.

### Teste selbst

Stift / leer → Stift bleibt, Anzahl leer; URL enthält keine Eingabedaten.

Leer / 0 → Artikel leer, 0 bleibt; beide leer → zwei Meldungen.

Stift / 2 → Vormerkung; danach fehlerhaft senden → alte Vormerkung unverändert.

## 09 Einen Entwurf in zwei Schritten bestätigen

Schritt 1: **titel** · Schritt 2: **Checkbox bestaetigt (ja)**

Session-Daten zwischen zwei Formularschritten verwenden

1. Prüfe den Titel aus dem ersten POST-Formular. Speichere nur gültigen Text als Session-Entwurf und leite zu `bestaetigen.php` weiter.
2. Die zweite Seite liest den Titel nur aus der Session. Ohne gültigen Entwurf führt sie mit einer Fehlermeldung zum ersten Formular zurück.
3. Prüfe die Bestätigung serverseitig: `bestaetigt` muss gesetzt, ein Textwert und genau ja sein. Ohne Haken bleibt der Entwurf erhalten und die zweite Seite zeigt einen Fehler.
4. Nach gültiger Bestätigung übernimmst du den Titel als Auftrag und entfernst den Entwurf. Die Startseite zeigt die Bestätigung. Das ist nur eine Ablaufübung, kein Login und kein Druckauftrag.

### Teste selbst

Titel „Übungsblatt“ → zweite Seite zeigt denselben Titel ohne URL-Parameter.

Ohne Haken senden → Meldung; Entwurf bleibt; mit Haken → Bestätigung.

Zweite Seite ohne Entwurf direkt öffnen → kontrolliert zum ersten Schritt zurück.

## 10 Die gesamte Übungs-Session beenden

POST · **alias** · **format (kompakt / gross)** · **Aktion beenden**

Session-Daten, PHP-Array und Browser-Cookie unterscheiden

1. Prüfe und speichere Alias sowie Format wie gewohnt. Verwende für Formularfehler den Session-Rückweg aus Aufgabe 8.
2. Ergänze die POST-Aktion `beenden`. Sie verlangt keine neuen Feldwerte. Leere sämtliche Werte dieser Session, entferne ihr Cookie mit passendem Namen und Pfad und zerstöre die serverseitigen Session-Daten.
3. Leite danach auf die vorbereitete `ende.html` weiter. Diese Seite darf keine neue Session starten. Andere Aufgabenordner dürfen nicht beeinflusst werden.
4. Öffne anschließend `index.php` erneut: Jetzt entsteht eine neue leere Session. Prüfe im Browser den Cookie-Zustand vor und nach der Beenden-Aktion.

### Teste selbst

Alias / Groß speichern → Werte vorhanden; Beenden → Cookie weg.

F5 auf `ende.html` → keine neue Session; Link zu `index.php` → neuer leerer Stand.

Alte Werte erscheinen nicht wieder; nur ein Eintrag `per unset()` wäre nicht dasselbe.

# Merkhilfe

Bausteine und Denkregeln – keine Aufgabenlösungen.

## Wer macht was?

PHP lädt nach `session_start()` die zugehörigen Nutzdaten. Die ID kommt hier über ein Cookie. `$_SESSION` ist im aktuellen Skript veränderbar; gespeichert wird normalerweise am Skriptende. [1–3]

## Eine Zuweisung wirkt sofort

Ein neuer Wert in `$_SESSION` ist schon im selben PHP-Aufruf lesbar. Für eine zweite Seite wird die Session dort erneut gestartet. Der Browser sendet dabei die ID, nicht das gesamte Session-Array. [1, 2]

## Ein Wert, alle Werte oder ganz beenden?

`unset($_SESSION["x"])` entfernt einen Schlüssel. `$_SESSION = []` leert die Werte, behält aber zunächst die Session. `session_destroy()` entfernt Serverdaten, nicht von allein das PHP-Array oder Cookie. Zum vollständigen Beenden die Schritte kombinieren. [1, 4]

## Typen und die Zahl 0

POST liefert hier Textwerte; nach Prüfung kannst du Zahlen als `int` speichern. Deshalb beim Lesen den erwarteten Session-Typ prüfen. `empty()` allein eignet sich nicht, um die erlaubte Zahl 0 von einer fehlenden Eingabe zu unterscheiden.

## Einmalmeldung ist ein eigenes Muster

Die Meldung aus der Session in eine lokale Variable kopieren, ihren Session-Eintrag entfernen und die lokale Variable anzeigen. Beim nächsten Aufruf ist die Meldung weg. Der fachliche Speicherstand bleibt bestehen.

## Session-ID und Lebensdauer

Keine echte Session-ID in HTML, URL oder Formularfelder übernehmen. Cookie-Lebensdauer 0 bedeutet nicht „sofort löschen“. Browser-Wiederherstellung und serverseitige Aufbewahrung sind getrennt; Browser schließen ist kein verlässlicher Logout-Test. [5, 6, 10]

## So testest du

Zweiten Tab und ein getrenntes Browserprofil vergleichen. Fehlerhafte und fehlende Felder senden. Nach POST die 303-Antwort und den folgenden GET prüfen. Nach dem vollständigen Beenden auf der statischen Abschlussseite bleiben: Sie startet keine neue Session. [1, 9]

## Technische Referenzen

[1] [PHP: Session-Grundlagen](#)

[3] [PHP: \\$\\_SESSION](#)

[5] [PHP: Session-Sicherheit](#)

[7] [PHP: session\\_write\\_close\(\)](#)

[9] [MDN: HTTP 303](#)

[2] [PHP: session\\_start\(\)](#)

[4] [PHP: session\\_destroy\(\)](#)

[6] [PHP: Cookie-Parameter](#)

[8] [PHP: header\(\)](#)

[10] [MDN: HTTP-Cookies](#)

Neue Übungen im Aufbau des Cookie-Pakets. Technische Regeln: PHP-Handbuch und MDN; vollständige Quellen und Zusatzhinweise in der Webvariante. Stand: 01.10.2026.